

论软件架构的演化与维护

⚠ 关于本文中的数字 本文的量化数据（遗留模块数量、改造周期、缺陷下降幅度等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字（表数量、接口数、配置值）**已从项目源码核实，可直接使用；**业务规模类数字（门店数、用户数、业务量、改善幅度）**在项目资料中尚未确定，属示例值，背诵前请替换**为你的真实数据**。文末附有「待替换数字清单」，便于逐项核对。

试题

论软件架构的演化与维护

软件架构并非一成不变，随着业务需求的变化、技术栈的更新以及运行环境的发展，软件架构需要持续演化与维护。架构演化既要满足新的业务与技术诉求，又要保证既有功能的稳定运行，因此需要在演进与稳定之间取得平衡。架构演化通常会面临遗留系统难以替换、新旧技术栈长期并存、演化过程中的兼容性保障等问题。

请围绕“软件架构的演化与维护”论题，依次从以下三个方面进行论述：

1. 简要叙述你参与维护和演化的软件系统，以及你所担任的工作；
2. 详细论述你在该系统中如何规划架构演化的路径，以及如何控制演化过程中的风险；
3. 分析你在演化过程中遇到的问题及解决措施。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了软件架构演化与维护的实践。该平台需要长期兼容集团基于旧版 AWS SDK 开发的既有系统，同时要从单体结构向微服务架构演进。我在项目中担任系统架构师，负责演化路径的规划、遗留系统的兼容方案设计以及演化风险的管控。针对遗留技术栈无法一次性重写、新旧版本依赖长期并存、接口契约在演化中容易漂移三大挑战，我采用了三条演化策略：以契约先行稳定接口边界，使内部实现可以自由替换而调用方无感；以渐进替换代替整体重写，按业务域逐个迁移而非一次性切换；以防腐层隔离旧技术栈，把历史包袱约束在有限范围内。同时，我通过统一版本管理插件把第三方依赖这一隐式契约显式化，并建立了以构建可复现性为核心的演化风险管控机制。演化实施后，接口一致性缺陷下降约 85%，构建产物实现完全可复现，因依赖版本漂移导致的运行时故障不再发生。

正文

一、项目背景与演化动因

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔。我在项目中担任系统架构师，负责演化路径的规划、遗留系统的兼容方案设计以及演化风险的管控。

平台的建设并非从零开始，而是在集团既有的系统格局中生长出来的。这决定了我们的架构演化必须面对三个既成事实。

第一是遗留技术栈的长期存在。 集团既有系统中有相当规模的代码基于旧版 AWS SDK 开发，这些系统的业务逻辑仍在正常运行，服务着集团的其他业务线。它们无法被一次性重写，因为重写意味着业务中断、测试成本与回归风险都难以承受。

第二是单体结构的演化压力。 既有系统采用单体部署，其问题在业务量增长后日益明显：任何一种负载的峰值都会波及其他业务，一次变更需要整体回归，任何一个模块的内存泄漏都会拖垮整个进程。平台的伸缩特征差异极大——预约受理是随营业时间波动的交互型负载，成本文件生成是夜间的计算密集型批量负载，数据同步则是持续低频的事件型负载。用单体承载这三类负载，必然导致资源的巨大浪费与相互干扰。

第三是多技术栈协作的现实。 平台需要同时为 Java 内部服务、Go 周边平台与 PHP 既有系统三类调用方提供服务，而这三类调用方的构建工具链、依赖管理方式与发布节奏完全不同。

面对这三个事实，我首先明确了一条原则：**架构演化不是推倒重来，而是在保证业务连续性的前提下逐步替换。**任何需要“停机切换”的方案，都不会被业务方接受，也不应该被架构师提出。

二、演化路径的规划

基于上述原则，我规划了三条并行的演化策略。

策略一：契约先行，稳定接口边界。

这是三条策略中最根本的一条。演化过程中最容易出问题的环节，往往是调用方与实现方被同时改动——一旦双方都需要变更，就必须协同发布，而协同发布的窗口极难把握，且一旦出现问题时无法判断是哪一方的责任。

因此我把平台的接口契约独立到规格模块中，使其不依赖任何业务实现模块。契约按服务定义、请求消息、响应消息、枚举与可复用消息五层组织，由同一份契约驱动 Java、Go、PHP 三语言客户端与 OpenAPI 文档的生成。这样做带来的关键收益是：**接口边界一旦稳定，边界以内的实现就可以自由替换**——内部从单体拆分为服务、从容器改为函数、从一种存储迁移到另一种存储，调用方都无需感知。契约成了演化过程中的“稳定面”，而全部变化都发生在“不稳定面”之内。

在契约本身的演化上，我遵循向后兼容的原则：新增字段而非修改字段，新增接口而非修改接口语义。对于确实需要破坏性变更的场景，则通过并行提供新旧两个版本来过渡，而不是直接替换。

策略二：渐进替换，按业务域逐个迁移。

对于从单体向服务化演进的部分，我没有采用“一次性重写”的方式，而是按业务域逐个迁移。每个业务域的迁移都遵循同样的步骤：先在契约中定义该业务域的接口；再把该业务域的逻辑从单体中抽离，实现为独立服务；随后把调用方逐个切换到新服务，切换期间新旧两条路径并存；最后在确认新路径稳定后，下线单体中的旧实现。

这一策略的核心价值在于**把一次大风险拆成若干次小风险**。每一次迁移的范围有限、影响可控、可独立回滚；即使某次迁移出现问题，也只影响该业务域，不会波及全平台。同时，拨离过程中暴露出的模块耦合问题，会自然地推动我们进一步理清边界——很多在原单体中看似清晰的模块，一旦尝试独立部署，才会发现其与周边模块的隐藏依赖。

对于平台中新增的能力，则直接以服务形态实现，不再往单体中增加代码。这样，单体的规模只减不增，其复杂度随时间推移而收敛，而不是继续膨胀。

策略三：防腐层，隔离旧技术栈。

对于必须与旧版 SDK 共存的场景，我没有要求全平台统一到新版 SDK——这既不现实，也会把旧版本的问题扩散到新代码中。相反，我在新旧之间建立了一层防腐层：把使用旧版 SDK 的代码收敛到明确的边界内——本项目

中主要是对象存储与安全令牌服务这两项能力仍在旧版 SDK 上——边界之外的新代码一律使用新版 SDK；跨边界的交互通过平台内部的统一抽象完成，而不是让新版代码直接依赖旧版 API 的类型。

这样做的结果是，历史包袱被约束在有限的范围内，其影响不会扩散到新写的代码。同时，这个边界也为将来的清理提供了明确的范围——当旧系统最终下线时，只需删除防腐层内的代码，而不需要在全平台范围内搜索旧 API 的调用。

三、演化风险的控制

控制一：把隐式契约显式化。

演化过程中出现的一次严重故障，让我重新认识了“契约”的范围。平台初期各模块独立声明依赖版本，部分组件采用了动态版本区间。一次依赖升级后，protobuf 官方公共类型库被解析到一个候选版本，导致 gRPC 服务启动直接失败。

排查后我意识到，我们虽然统一了业务接口的契约，却忽略了**第三方依赖版本这一隐式契约**——两个模块即使业务接口完全一致，也可能因为底层库版本不同而无法互通。而且这种不一致在编译期完全不可见，只在运行时才暴露。

因此我建立了独立的统一版本管理插件，各模块通过插件接入统一的版本源，关键组件使用强制版本策略锁定。改造后，同一提交在任何机器、任何时间构建出的产物完全一致，构建具备了可复现性。这项改造的价值在演化过程中尤为突出：它使“演化前后的行为差异”可以被明确归因，而不会与“依赖版本漂移导致的差异”混在一起。

控制二：以构建结果界定变更影响。

在多团队协作的演化场景下，一次接口变更的影响范围往往需要人工梳理——哪些调用方会受影响、哪个语言的产物需要重新发布。人工梳理难免遗漏，而遗漏的代价是运行时的集成失败。

由于契约是单一来源且产物由构建流水线自动生成，变更的影响范围可以从构建结果中直接读出。这使变更评审从“依赖经验判断”变为“以构建产物为准”，评审效率与准确性同时提升。

控制三：可回滚优先于可修复。

演化过程中，我要求每一项变更都必须具备回滚路径。具体而言：接口层面保持向后兼容，使新版本可以随时退回旧版本；部署层面通过容器镜像的版本管理，使任何一次发布都可以快速回退到上一个稳定版本；数据层面则通过新增字段而非修改字段的方式，保证新旧代码可以同时读取同一份数据。

这一原则的出发点是：**在演化过程中，“能退回”比“能修好”更重要。**线上的问题往往需要在极短时间内响应，而修复需要定位、编码、测试、发布的完整周期，回滚只需要一次镜像切换。

四、演化过程中遇到的问题及解决措施

问题一：跨语言接口契约的一致性维护。项目初期各端曾各自根据文档维护接口定义，导致同一字段在不同端的命名与类型出现差异，直到联调阶段才被发现。这是典型的演化失控——接口在多方手中各自漂移。我的判断是，根源不是开发人员疏忽，而是**契约缺乏单一来源**这一架构缺陷。因此我把接口定义全部收敛到规格模块，使接口变更必须通过该模块提交，并把三语言产物的生成纳入构建流水线，让不一致问题从“运行时联调发现”提前到“构建期暴露”。改造后，接口一致性缺陷较改造前下降约 85%。

问题二：依赖版本漂移引发的运行时故障。如前所述，protobuf 公共类型库被解析到候选版本，导致 gRPC 服务启动直接失败。这一故障在演化过程中的危害尤其大，因为它发生在新旧代码并存的状态下，容易被误判为演化本身引入的问题。解决方案即统一版本管理插件与强制版本策略。

问题三：旧版与新版 SDK 共存的长期成本。 双栈并存的状态在初期是必要的，但随着时间推移，其代价逐渐显现：开发人员需要同时熟悉两套 API，依赖树中存在版本冲突的风险，构建体积与构建时间都有增加。我对此的处理方式是**接受这一成本，但为其设定明确的收敛路径**：把旧版 SDK 的使用范围约束在防腐层内，并禁止新增使用；在每一次业务迭代中，如果某个模块因业务需要而改动，则顺带完成该模块的 SDK 迁移，而不是专门安排一次迁移任务。这种"搭车式迁移"的优点是迁移成本被摊薄到日常迭代中，不需要专门的项目预算与测试周期。需要客观说明的是，目前这一过渡状态尚未终结，长期维护成本依然存在。

问题四：演化中的功能回归。 在按业务域逐个迁移的过程中，曾出现过迁移后某个边界场景的行为与原实现不一致的情况。这类问题的根源在于，原实现中的某些行为是历史累积形成的，且并未被文档记录——典型的"隐性知识"。我的应对措施有两条：一是迁移前补充针对该业务域的自动化测试，把可识别的行为固化为测试用例；二是迁移采用新旧路径并存的方式，通过双跑比对来发现行为差异，而不是直接切换。这样，即使存在未被文档记录的隐性行为，也能在双跑阶段被发现。

五、总结与展望

平台演化实施至今已稳定运行约 8 个月，年可用率达到 99.95%，业务量较原系统提升约 40%。演化方面的成效主要体现在：接口一致性缺陷较改造前下降约 85%；构建产物实现完全可复现，因依赖版本漂移导致的运行时故障不再发生；单体的规模只减不增，复杂度随时间收敛而非膨胀。

目前仍存在三点不足。第一，旧版与新版云 SDK 并存的过渡状态尚未终结，长期维护成本依然存在；第二，契约的破坏性变更尚未实现自动化的兼容性检查，仍依赖人工评审；第三，防腐层内部的历史代码缺少测试覆盖，其行为主要依赖人工理解，这也使得清理工作需要格外谨慎。

后续计划引入契约兼容性校验工具，在构建阶段自动拦截破坏性变更；制定旧版 SDK 的最终收敛时间表；并逐步为防腐层内的代码补充测试，为最终的清理创造条件。

通过本项目，我对架构演化形成了三点认识。第一，演化的核心矛盾是"变"与"不变"的划分——架构师的任务是找到那个可以长期不变的稳定面（在本项目中是接口契约），并让变化都发生在它之后。第二，演化中最危险的不是技术难题，而是**未被记录的隐性行为**；因此"先补测试再迁移"不是保守，而是必需。第三，演化能力的上限取决于构建与部署能力——如果没有可复现的构建、可回滚的发布与自动化生成的契约产物，再好的演化策略也无从落地，这也印证了那句常被引用的话：架构演化的速度，受限于交付流水线的速度。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量	约 3 万笔	真实数量
接口一致性缺陷下降	约 85%	真实数据
业务量提升	约 40%	真实数据
稳定运行时长 / 可用率	约 8 个月 / 99.95%	真实指标
遗留 SDK 范围	对象存储 / 安全令牌服务两项	✔ 已核实

位置	示例值	请替换为
迁移的业务域数量	未给出（可补充）	真实数量

本文字数

摘要约 350 字，正文约 2900 字，合计约 3250 字。

可复用要点

- 「架构演化不是推倒重来，而是在保证业务连续性前提下逐步替换」 是开篇立场的标准表述
- 三条演化策略（契约先行稳定边界 / 渐进替换按域迁移 / 防腐层隔离旧栈）中，绞杀者模式与防腐层是有正式名称的架构模式，写出来能显著提升专业度
- 「能退回比能修好更重要」 是演化风险管控的精炼总结，非常有力
- 「搭车式迁移」 的做法（借业务改动顺带完成技术迁移）是真实可行的工程实践，比"安排专项迁移"更务实
- 「未被记录的隐性行为是演化中最危险的」 是本篇最有洞察力的判断
- 若题目问"遗留系统改造策略"，直接引用第二节的三条策略